- 空白开头
- 基础算法
  - 优先队列（堆）
  - 并查集
  - 快速幂模板
  - 多线程
  - 合并果子（堆）
  - 全排列
  - 约瑟夫（队列）
  - 马的遍历
  - 二分
  - 查找文献（dfs）
  - 前缀和函数
  - 排队接水（包含小数位的保留）
  - manacher算法（O（n）判断回文串）
  - 运算符重载
  - 快读快写模板
  - 三分
  - 字符串哈希（区分不同字符串）
  - KMP
  - Int_128
  - 哈希表
  - 字符串哈希
- DP
  - 采药dp
  - 状压DP
- 数学算法（含高精度）
  - 最大公约数 & 最小公倍数
  - 线性筛素数
  - 高精度+
  - 高精度-
  - 高精度X
  - 高精度/
  - 异或
  - 求逆元(1~N)
  - 差分数组
- 图论

    - 最小生成树模板
    - 最短路径算法（迪杰斯特拉）
    - 图的遍历
    - 树的直径
    - 树形DP
    - LCA
    - (最近公共祖先)
    - 拓扑排序
    - 强连通分量
    - Floyd（全源最短路）
- 数据结构 STL 特殊库函数
    - queue
    - priority_queue
    - set
    - vector
    - map
    - stack
    - pair
    - string
    - 线段树
        - 基础线段树
    - memset函数
- 数学小妙招

空白开头

```
#include<bits/stdc++.h>
#define endl '\n'
using namespace std;
void solve(){
    return ;
}
int main(){
   ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
   int _=1;
   cin>>_;
   while(t--){
      solve();
   }
   return 0;
}
```

基础算法

优先队列（堆）

P3378 【模板】堆

题目描述

给定一个数列，初始为空，请支持下面三种操作：

1. 给定一个整数 x，请将 x 加入到数列中。
2. 输出数列中最小的数。
3. 删除数列中最小的数（如果有多个数最小，只删除 1 个）。

输入格式

第一行是一个整数，表示操作的次数 n。接下来 n 行，每行表示一次操作。每行首先有一个整数 op 表示操作类型。

- 若 $op = 1$，则后面有一个整数 x，表示要将 x 加入数列。
- 若 $op = 2$，则表示要求输出数列中的最小数。
- 若 $op = 3$，则表示删除数列中的最小数。如果有多个数最小，只删除 1 个。

输出格式

对于每个操作 2，输出一行一个整数表示答案。

```
#include<bits/stdc++.h>
using namespace std;
const int M=1e6+10;
int n,x,k;
priority_queue<int,vector<int>,greater<int> >q;
int main(){
    cin>>n;
    int j=0;
    for(int i=0;i<n;i++){
        cin>>x;
        switch(x){
    case 1:
        cin>>k;            q.push(k);
        break;
    case 2:
        cout<<q.top()<<"\n";
        break;
    case 3:            q.pop();
        }
    }
    return 0;
}
```

并查集

P3367 【模板】并查集

题目描述

如题，现在有一个并查集，你需要完成合并和查询操作。

输入格式

第一行包含两个整数 N, M ,表示共有 N 个元素和 M 个操作。

接下来 M 行，每行包含三个整数 Z_i, X_i, Y_i 。

当 $Z_i = 1$ 时，将 X_i 与 Y_i 所在的集合合并。

当 $Z_i = 2$ 时，输出 X_i 与 Y_i 是否在同一集合内，是的输出 `Y` ；否则输出 `N` 。

输出格式

对于每一个 $Z_i = 2$ 的操作，都有一行输出，每行包含一个大写字母，为 `Y` 或者 `N`。

```
#include<bits/stdc++.h>
using namespace std;
const int M=2e5+10;
int num[M];
int z,x,y;
int find(int x){
    if(num[x]==x) return x;
    else{
        int t=find(num[x]);
        num[x]=t;
        return num[x];
    }
}
int main(){
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++) num[i]=i;
    while(m--){
        cin>>z>>x>>y;
        if(z==1) num[find(x)]=find(y);
        else {
            if(find(x)==find(y)) cout<<"Y"<<endl;
            else cout<<"N"<<endl;
        }
    }
    return 0;
}
```

快速幂模板

P1226 【模板】快速幂

输入格式

输入只有一行三个整数，分别代表 a, b, p。

输出格式

输出一行一个字符串 `a^b mod p=s`，其中 a, b, p 分别为题目给定的值， s 为运算结果。

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
```

```
int a,b,p;
int64_t powint(int base, int exp)
{
    int64_t result=1;
    while (exp)
    {
        if (exp & 1) result =1ll*result*base%p;
        exp >>= 1;
        base =1ll*base*base%p;
    }
    return result;
}
signed main(){
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    cin>>a>>b>>p;
    int ans=powint(a,b);
    ans %=p;
    printf("%d^%d mod %d=%d",a,b,p,powint(a,b));
    return 0;
}
```

多线程

P2392 kkksc03考前临时抱佛脚

题目背景

kkksc03 的大学生活非常的颓废，平时根本不学习。但是，临近期末考试，他必须要开始抱佛脚，以求不挂科。

题目描述

这次期末考试，kkksc03 需要考 4 科。因此要开始刷习题集，每科都有一个习题集，分别有 s_1, s_2, s_3, s_4 道题目，完成每道题目需要一些时间，可能不等（$A_1, A_2, \ldots, A_{s_1}$，$B_1, B_2, \ldots, B_{s_2}$，$C_1, C_2, \ldots, C_{s_3}$，$D_1, D_2, \ldots, D_{s_4}$）。

kkksc03 有一个能力，他的左右两个大脑可以同时计算 2 道不同的题目，但是仅限于同一科。因此，kkksc03 必须一科一科的复习。

由于 kkksc03 还急着去处理洛谷的 bug，因此他希望尽快把事情做完，所以他希望知道能够完成复习的最短时间。

输入格式

本题包含 5 行数据：第 1 行，为四个正整数 s_1, s_2, s_3, s_4。

第 2 行，为 $A_1, A_2, \ldots, A_{s_1}$ 共 s_1 个数，表示第一科习题集每道题目所消耗的时间。

第 3 行，为 $B_1, B_2, \ldots, B_{s_2}$ 共 s_2 个数。

第 4 行，为 $C_1, C_2, \ldots, C_{s_3}$ 共 s_3 个数。

第 5 行，为 $D_1, D_2, \ldots, D_{s_4}$ 共 s_4 个数，意思均同上。

dp写法

```
#include<bits/stdc++.h>
#define endl '\n'
using namespace std;
const int M=1e5+10;
int t[M];
int num[M];
int dp[M];
int ans=0;
void solve(){
        for(int i=1;i<=4;i++){
                cin>>t[i];
        }
        for(int i=1;i<=4;i++){
                int sum=0;
                for(int j=1;j<=t[i];j++){
                        cin>>num[j];
                        sum+=num[j];
                }
                memset(dp,0,sizeof(dp));
                dp[0]=1;
                for(int j=1;j<=t[i];j++){
                        for(int k=sum;k>=0;k--){
                                if(dp[k]){
                                        dp[k+num[j]]=1;
                                }
                        }
                }
                int temp=sum;
                int res;
                for(int j=0;j<=sum;j++){
                        if(dp[j] && temp>=abs(2*j-sum)){
                                temp=abs(2*j-sum);
                                res=max(j,sum-j);
                        }
                }
                ans+=res;
        }
        cout<<ans<<endl;
}
int main(){
        ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
        int _=1;
        //cin>>_;
        while(_--){
```

```
                solve();
        }
        return 0;
}
```

dfs写法

```
#include<bits/stdc++.h>
using namespace std;
const int M=25;
int num[M],res,n,sum,ans;
int lop[5];
void dfs(int k,int s){
        if(k>n){
                int l=s;
                int r=sum-l;
                ans =min(ans,max(l,r));
                return ;
        }
        dfs(k+1,s+num[k]);
        dfs(k+1,s);
        return ;
}
int main(){
        ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
        for(int i=1;i<=4;i++) cin>>lop[i];
        for(int i=1;i<=4;i++){
                sum=0,ans=1e9;
                n=lop[i];
                for(int j=1;j<=n;j++){
                        cin>>num[j];
                        sum+=num[j];
                }
                dfs(1,0);
                res+=ans;
        }
                cout<<res<<'\n';
        return 0;
}
```

合并果子（堆）

P1090 [NOIP 2004 提高组] 合并果子

题目描述

在一个果园里，多多已经将所有的果子打了下来，而且按果子的不同种类分成了不同的堆。多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子的重量之和。可以看出，所有的果子经过 $n-1$ 次合并之后， 就只剩下一堆了。多多在合并果子时总共消耗的体力等于每次合并所耗体力之和。

因为还要花大力气把这些果子搬回家，所以多多在合并果子时要尽可能地节省体力。假定每个果子重量都为 1 ，并且已知果子的种类 数和每种果子的数目，你的任务是设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

例如有 3 种果子，数目依次为 1 ， 2 ， 9 。可以先将 1 、 2 堆合并，新堆数目为 3 ，耗费体力为 3 。接着，将新堆与原先的第三堆合并，又得到新的堆，数目为 12 ，耗费体力为 12 。所以多多总共耗费体力 $=3+12=15$ 。可以证明 15 为最小的体力耗费值。

输入格式

共两行。 第一行是一个整数 $n(1 \le n \le 10000)$ ，表示果子的种类数。

第二行包含 n 个整数，用空格分隔，第 i 个整数 $a_i(1 \le a_i \le 20000)$ 是第 i 种果子的数目。

输出格式

一个整数，也就是最小的体力耗费值。输入数据保证这个值小于 2^{31} 。

```
#include<bits/stdc++.h>
using namespace std;
const int M=1e5+10;
int n,x,ans;
priority_queue<int,vector<int>,greater<int> >q;
int main(){
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    cin>>n;
    for(int i=0;i<n;i++){
        cin>>x;
        q.push(x);
    }
    while(q.size()>=2){
        int x1=q.top();
        q.pop();
        int x2=q.top();
        q.pop();
        ans+=x1+x2;
        q.push(x1+x2);
    }
    cout<<ans<<endl;
    return 0;
}
```

全排列

```
#include<bits/stdc++.h>
using namespace std;
int n,num[15],ans[15];
void print(){
    for(int i=1;i<=n;i++){
        printf("%5d",ans[i]);
    }
    cout<<"\n";
    return ;
}
void dfs(int k){
    if(k>n){
        print();
        return ;
    }
    for(int i=1;i<=n;i++){
        if(!num[i]){
            num[i]=1;
            ans[k]=i;
            dfs(k+1);
            num[i]=0;
        }
    }
    return ;
}
int main(){
    cin>>n;
    dfs(1);
    return 0;
}
```

约瑟夫（队列）

```
#include<bits/stdc++.h>
using namespace std;
queue<int>q;
int main(){
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        q.push(i);
    }
    int i=0;
    while(!q.empty()){
        i++;
        if(i==m){
```

```
            cout<<q.front()<<" ";
            q.pop();
            i=0;
        }
        else{
            q.push(q.front());
            q.pop();
        }
    }
    return 0;
}
```

马的遍历

P1443 马的遍历

题目描述

有一个 $n \times m$ 的棋盘，在某个点 (x, y) 上有一个马，要求你计算出马到达棋盘上任意一个点最少要走几步。

输入格式

输入只有一行四个整数，分别为 n, m, x, y。

输出格式

一个 $n \times m$ 的矩阵，代表马到达某个点最少要走几步（不能到达则输出 -1）。

```
#include<bits/stdc++.h>
using namespace std;
int MAP[405][405];
int mx[10]={ 1,1,-2, 2,-1,-1,-2,2};
int my[10]={-2,2,-1,-1,-2, 2, 1,1};
int n,m,dx,dy,x,y;
queue<pair<int,int>>q;
```

```
int main(){
    memset(MAP,-1,sizeof(MAP));
    cin>>n>>m>>dx>>dy;
    MAP[dx][dy]=0;
    q.push(make_pair(dx,dy));
    while(q.size()){
        x=q.front().first;
        y=q.front().second;
```

```
        for(int i=0,newx,newy;i<8;i++){
            newx=x+mx[i];
            newy=y+my[i];
            if(newx>=1&&newx<=n&&newy>=1&&newy<=m&&MAP   [newx][newy]==-1){
  MAP[newx][newy]=MAP[x][y]+1;
 q.push(make_pair(newx,newy));
            }
        }
        q.pop();
    }
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            printf("%-5d",MAP[i][j]);
        }
        cout<<'\n';
    }
    return 0;
}
```

二分

```
//二分的一般写法
#include<bits/stdc++.h>
using namespace std;
const int M=1e6+10;
long long int num[M];
int n,m;
int xia(int x){
    int l=1,r=n;
    while(l<r){
        int mid=l+(r-l)/2;
        if(num[mid]>=x)r=mid;
        else l=mid+1;
    }
    if(num[l]==x) return l;
    else return -1;
}
int main(){
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++){
        scanf("%d",&num[i]);
    }
    int x;
    for(int i=0;i<m;i++){
        scanf("%d",&x);
        printf("%d ",xia(x));
    }
    return 0;

}

//利用STL的二分函数lower_bound和upper_bound实现
```

```
#include <iostream>

#include <algorithm>

using namespace std;

int a[8] = {1, 2, 4, 4, 5, 8, 10, 22};

int main(){

    auto x = lower_bound(a, a + 8, 4) ;//>=

auto y = upper_bound(a, a + 8, 4) ;//>

//获得的是数字的指针

//加*即可获得数字本身

    bool z = binary_search(a, a + 8, 4);//查找

    cout<<*x<<" "<<*y<<" "<<z<<endl;

    x = lower_bound(a, a + 8, 7) - a;//>=

y = upper_bound(a, a + 8, 7) - a;//>

//获得下标

    z = binary_search(a, a + 8, 7);//查找

    cout<<x<<" "<<y<<" "<<z<<endl;

    return 0;

}
```

注：若减去数列则可以获得下标，否则得到数字；

若找不到，则返回end，即数组/容器最后一个元素的下一个元素

如果要在结构体中使用lower_bound和upper_bound，把我们需要查找的数封装成一个结构体。然后才可以在结构体重进行查找。即使我们只需要针对某一维进行查找，也需要把整个结构体构造出来。
如下：

```
1.重载
struct Node {
    int a, b;
    bool operator<(const Node& o) const {
        return a < o.a;  // 按 a 排序/查找
    }
};
vector<Node> v = {{1,2}, {3,4}, {5,6}};
sort(v.begin(), v.end());  // 用 operator< 排序
auto it = lower_bound(v.begin(), v.end(), Node{4, 0});
2.cmp
struct Node {
```

```
    int a, b;
};
bool cmpA(const Node& x, const Node& y) { return x.a < y.a; }
vector<Node> v = {{5,1}, {3,4}, {1,9}};
sort(v.begin(), v.end(), cmpA);  // 按 a 排: {1,9}, {3,4}, {5,1}
auto it = lower_bound(v.begin(), v.end(), Node{4,0}, cmpA);  // 找 a>=4
```

查找文献（dfs）

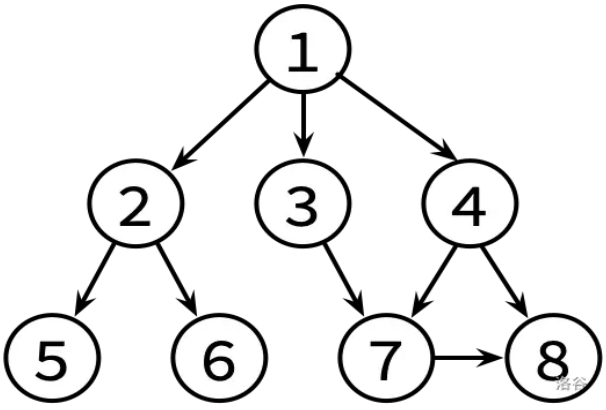
P5318 【深基18.例3】查找文献

题目描述

小 K 喜欢翻看洛谷博客获取知识。每篇文章可能会有若干个（也有可能没有）参考文献的链接指向别的博客文章。小 K 求知欲旺盛，如果他看了某篇文章，那么他一定会去看这篇文章的参考文献（如果他之前已经看过这篇参考文献的话就不用再看它了）。

假设洛谷博客里面一共有 $n(1 \le n \le 10^5)$ 篇文章（编号为 1 到 n）以及 $m(1 \le m \le 10^6)$ 条参考文献引用关系。目前小 K 已经打开了编号为 1 的一篇文章，请帮助小 K 设计一种方法，使小 K 可以不重复、不遗漏的看完所有他能看到的文章。

这边是已经整理好的参考文献关系图，其中，文献 $X \to Y$ 表示文章 X 有参考文献 Y。不保证编号为 1 的文章没有被其他文章引用。



请对这个图分别进行 DFS 和 BFS，并输出遍历结果。如果有很多篇文章可以参阅，请先看编号较小的那篇（因此你可能需要先排序）。

输入格式

共 $m+1$ 行，第 1 行为 2 个数，n 和 m，分别表示一共有 $n(1 \le n \le 10^5)$ 篇文章（编号为 1 到 n）以及 $m(1 \le m \le 10^6)$ 条参考文献引用关系。

接下来 m 行，每行有两个整数 X, Y 表示文章 X 有参考文献 Y。

输出格式

共 2 行。

第一行为 DFS 遍历结果，第二行为 BFS 遍历结果。

```
#include<bits/stdc++.h>
using namespace std;
const int M=1e6+10;
int n,m,x,y;
vector<int>f[M];
int vis[M];
void dfs(int x){
    vis[x]=1;
    cout<<x<<' ';
    for(auto to:f[x]){if(!vis[to]) dfs(to);    }
    return ;
}
int main(){
    cin>>n>>m;
    while(m--){
        cin>>x>>y;
        f[x].push_back(y);
    }
    for(int i=1;i<=n;i++){
        sort(f[i].begin(),f[i].end());
    }
    dfs(1);
    cout<<'\n';
    queue<int>q;
    for(int i=0;i<=n;i++)vis[i]=0;
    q.push(1);
    vis[1]=1;
    while(!q.empty()){
        int t=q.front();
        q.pop();
        cout<<t<<' ';
        for(int to:f[t]){
            if(!vis[to]){
                q.push(to);
                vis[to]=1;
            }}}
    return 0;
}
```

前缀和函数

std::partial_sum(a,a+6,res1); 需要注意的是，如果想计算到 `a[i]`处的部分和，`__last`参数必须传入 `a+i+1`。

```
#include <iostream>

using namespace std;

int N, A[10000], B[10000];

 int main() { cin >> N; for (int i = 0; i < N; i++) { cin >> A[i]; }

// 前缀和数组的第一项和原数组的第一项是相等的。

B[0] = A[0]; for (int i = 1; i < N; i++) {

// 前缀和数组的第 i 项 = 原数组的 0 到 i-1 项的和 + 原数组的第 i 项。

B[i] = B[i - 1] + A[i];

 }

 for (int i = 0; i < N; i++) {

cout << B[i] << " ";

}

return 0; }
```

排队接水（包含小数位的保留）

```
#include<bits/stdc++.h>
using namespace std;
const int M=1e3+10;
int sum[M];
struct peo{
    int a;
    int b;
}people[M];
bool check(peo x,peo y){
    return x.b<y.b;
}
void solve(){
    int n;
    double num=0.0;
    cin>>n;
    for(int i=1;i<=n;i++){
        people[i].a=i;
        cin>>people[i].b;
    }
    sort(people+1,people+1+n,check);
    for(int i=1;i<=n;i++){
        num+=(n-i)*people[i].b;
```

```
        cout<<people[i].a<<' ';
    }
    cout<<'\n'<<fixed<<setprecision(2)<<num/n;
    return ;
}
int main(){
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    solve();
    return 0;
}
```

manacher算法（O（n）判断回文串）

```
#include<cstdio>
#include<cstring>
#include<algorithm>
#include<iostream>
using namespace std;
const int maxn = 1e6 + 5;
char s[maxn * 2], str[maxn * 2];
int Len[maxn * 2], len;
void getstr() {//重定义字符串
    int k = 0;
    str[k++] = '@';//开头加个特殊字符防止越界
    for (int i = 0; i < len; i++) {
        str[k++] = '#';
        str[k++] = s[i];
    }
    str[k++] = '#';  len = k;
    str[k] = 0;//字符串尾设置为0，防止越界
}
int manacher() {
    int mx = 0, id;//mx为最右边，id为中心点
    int maxx = 0;
    for (int i = 1; i < len; i++) {
        if (mx > i) Len[i] = min(mx - i, Len[2 * id - i]);//判断当前点超没超过mx
        else Len[i] = 1;//超过了就让他等于1，之后再进行查找
        while (str[i + Len[i]] == str[i - Len[i]]) Len[i]++;//判断当前点是不是最长回
文子串，不断的向右扩展
        if (Len[i] + i > mx) {//更新mx
            mx = Len[i] + i;
            id = i;//更新中间点
            maxx = max(maxx, Len[i]);//最长回文字串长度
        }
    }
    return (maxx - 1);
}
int main() {
        scanf("%s", s);
                len = strlen(s);
                getstr();
```

```
                printf("%d\n",manacher());
                return 0;
}
```

运算符重载

```
Point operator+(const Point &a,const Point &b)
{//__类外重载,运算符重载函数作为类的友元函数_
Point ret; ret.x = a.x + b.x; ret.y = a.y + b.y; return ret;
}
```

快读快写模板

```
#include<bits/stdc++.h>
using namespace std;
inline int read(){
    int x=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9'){
        if(ch=='-')
            f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9'){
        x=(x<<1)+(x<<3)+(ch^48);
        ch=getchar();
    }
    return x*f;
}
inline void write(int x)
{
    if(x<0){
        putchar('-');
                x=-x;
        }
    if(x>9)
                write(x/10);
    putchar(x%10+'0');
}

int main(){
        int n;
        n=read();
        for(int i=1;i<=n;i++){
                int a;
                a=read();
```

```
                write(a);
                putchar(' ');
        }
        return 0;
}
```

三分

给定n个二次函数 $f_1(x), f_2(x), f_3(x)....f_n(x)$，（均形如$ax^2+bx+c$），设$F(x)=max\{f_1(x), f_2(x), f_3(x)....f_n(x)\}$，求$F(x)$在区间$[0, 1000]$上的最小值。

```
#include <bits/stdc++.h>
constexpr int N = 1e4 + 10;
constexpr double eps = 1e-10;
int n;
double a[N], b[N], c[N];
double f(double x)
{
  double res = -1e9;
  for (int i = 1; i <= n; i++)
  {
  res=std::max(res,a[i]*x*x+b[i]*x+c[i]);
  }
    return res;
}
void solve()
{
    std::cin >> n;
    for (int i = 1; i <= n; i++)
    {
        std::cin >> a[i] >> b[i] >> c[i];
    }
    double l = 0, r = 1000;
    while (r - l > eps)
    {
        double mid1 = (2 * l + r) / 3;
        double mid2 = (l + 2 * r) / 3;
        if (f(mid1) > f(mid2))
        {
            l = mid1;
        }
        else
        {
            r = mid2;
        }
    }
    std::cout << std::fixed << std::setprecision(4) << f(l) << '\n';
}
signed main()
{
```

```
    std::ios::sync_with_stdio(0);
    std::cin.tie(0);
     int _;
    std::cin >> _;
    while (_--)
        solve();
    return 0;
}
```

字符串哈希（区分不同字符串）

给定N个字符串（第i个字符串长度为M_1，字符串内包含数字、大小写字母，大小写敏感），请求出N个字符串中共有多少个不同的字符串

```
#include <bits/stdc++.h>
constexpr int N = 2e3 + 10;
constexpr int maxn = 1e4 + 10;
constexpr int mod = 1e9 + 7;
char s[N];
int a[maxn];
void hashchar(int ind)
{ _//_ _对第_ _ind_ _个字符串进行字符串哈希
    int base = 130;
    int ans = 0;
    for (int i = 0; s[i];i++)
    {
        ans = (ans + s[i]) * base;
        ans %= mod;
    }
    a[ind] = ans;
}
void solve()
{
    int n;
    std::cin >> n;
    for (int i = 1; i <= n; i++)
    {
        std::cin >> s;
        hashchar(i);
    }
    std::sort(a + 1, a + n + 1);
    int cnt = 0;
    for (int i = 1; i <= n;i++)
    {
        cnt += (a[i] != a[i - 1]);
    }
    std::cout << cnt << std::endl;
}
signed main()....
```

KMP

```
#include <bits/stdc++.h>
constexpr int N = 1e6 + 10;
std::string S, P; //_ _分别表示主串与模式串（在主串中寻找模式串）
int n, m; _//_ _分别表示模式串与主串的长度
int ne[N];
void init()
{
    std::cin >> S >> P;
    S = " " + S, P = " " + P;
    n = P.length() - 1;
    ne[1] = 0;
    for (int i = 2, j = 0; i <= n; i++)
    {
        while (j && P[i] != P[j + 1])
            j = ne[j];
        if (P[i] == P[j + 1])
            j++;
        ne[i] = j;
    }
}
void KMP()
{
    m = S.length() - 1;
    for (int i = 1, j = 0; i <= m; i++)
    {
        while (j && S[i] != P[j + 1])
            j = ne[j];
        if (S[i] == P[j + 1])
            j++;
        if (j == n)
            std::cout << i - n + 1 << '\n';
    }
    for (int i = 1; i <= n; i++)
        std::cout << ne[i] << " ";
    std::cout << '\n';
}
signed main()...
```

Int_128

```
#include <bits/stdc++.h>

using i128 = __int128_t;
std::ostream &operator<<(std::ostream &os, __uint128_t n)
{
    if (n > 9)  os << n / 10;
```

```
    os << (int)(n % 10);
    return os;
}
std::ostream &operator<<(std::ostream &os, __int128_t n)
{
    if (n < 0)
    {
        os << '-';
        n = -n;
    }
    return os << (__uint128_t)n;
}
void solve()
{
    i128 a = 123456789;
    std::cout << a * a * a << '\n';
}
signed main()
{
    std::ios::sync_with_stdio(0);
    std::cin.tie(0);
    int _ = 1;
//_ _std::cin_ _>>_ __;
    while (_--)
    solve();
    return 0;
}
```

哈希表

```
struct hash_map {  //_ _哈希表模板
        struct data {
        long long u;
        int v, nex;
    };  //_ _前向星结构
        data e[SZ << 1];  //_ _SZ_ _是_ _const_ _int_ _表示大小
    int h[SZ], cnt;

    int hash(long long u) { return (u % SZ + SZ) % SZ; }

    //_ _这里使用_ _(u_ _%_ _SZ_ _+_ _SZ)_ _%_ _SZ_ _而非_ _u_ _%_ _SZ_ _的原因是
    //_ _C++_ _中的_ _%_ _运算无法将负数转为正数
        int& operator[](long long u) {
        int hu = hash(u);  //_ _获取头指针
        for (int i = h[hu]; i; i = e[i].nex)
           if (e[i].u == u) return e[i].v;
        return e[++cnt] = data{u, -1, h[hu]}, h[hu] = cnt, e[cnt].v;
    }

    hash_map() {
        cnt = 0;
        memset(h, 0, sizeof(h));
```

```
    }
};
```

字符串哈希

(单模)

```
using std::string;
constexpr int M = 1e9 + 7;
constexpr int B = 233;

using ll = long long;

int get_hash(const string& s) {
    int res = 0;
    for (int i = 0; i < s.size(); ++i) {
        res = ((ll)res * B + s[i]) % M;
    }
    return res;
}

bool cmp(const string& s, const string& t) {
    return get_hash(s) == get_hash(t);
}
```

(双模)

```
using ull = unsigned long long;
ull base = 131;
ull mod1 = 212370440130137957, mod2 = 1e9 + 7;

ull get_hash1(std::string s) {
    int len = s.size();
    ull ans = 0;
    for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod1;
    return ans;
}

ull get_hash2(std::string s) {
    int len = s.size();
    ull ans = 0;
    for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod2;
    return ans;
}

bool cmp(const std::string s, const std::string t) {
    bool f1 = get_hash1(s) != get_hash1(t);
```

```
    bool f2 = get_hash2(s) != get_hash2(t);
    return f1 || f2;
}
```

DP

采药dp

P1048 [NOIP 2005 普及组] 采药

题目描述

辰辰是个天资聪颖的孩子，他的梦想是成为世界上最伟大的医师。为此，他想拜附近最有威望的医师为师。医师为了判断他的资质，给他出了一个难题。医师把他带到一个到处都是草药的山洞里对他说：“孩子，这个山洞里有一些不同的草药，采每一株都需要一些时间，每一株也有它自身的价值。我会给你一段时间，在这段时间里，你可以采到一些草药。如果你是一个聪明的孩子，你应该可以让采到的草药的总价值最大。”

如果你是辰辰，你能完成这个任务吗？

输入格式

第一行有 2 个整数 T（$1 \leq T \leq 1000$）和 M（$1 \leq M \leq 100$），用一个空格隔开，T 代表总共能够用来采药的时间，M 代表山洞里的草药的数目。

接下来的 M 行每行包括两个在 1 到 100 之间（包括 1 和 100）的整数，分别表示采摘某株草药的时间和这株草药的价值。

输出格式

输出在规定的时间内可以采到的草药的最大总价值。

```
#include<bits/stdc++.h>
using namespace std;
const int M=105;
int ti[M],v[M];
int dp[10005][10005];
int main(){
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    int t,m;
    cin>>t>>m;
    for(int i=1;i<=m;i++){
```

```
        cin>>ti[i]>>v[i];
    }
    for(int i=1;i<=m;i++){
        for(int j=1;j<=t;j++){
            if(j<ti[i]) dp[i][j]=dp[i-1][j];
            else dp[i][j]=max(dp[i-1][j],dp[i-1][j-ti[i]]+v[i]);
        }
    }
    cout<<dp[m][t]<<endl;
    return 0;
}
```

状压DP

P1559 运动员最佳匹配问题

题目描述

羽毛球队有男女运动员各 n 人。给定 2 个 $n \times n$ 矩阵 P 和 Q。$P_{i,j}$ 是男运动员 i 和女运动员 j 配对组成混合双打的男运动员竞赛优势；$Q_{i,j}$ 是女运动员 i 和男运动员 j 配合的女运动员竞赛优势。

但是，由于技术配合和心理状态等各种因素影响，$P_{i,j}$ 不一定等于 $Q_{j,i}$。男运动员 i 和女运动员 j 配对组成混合双打的男女双方竞赛优势为 $\boldsymbol{P_{i,j} \times Q_{j,i}}$。

现在，请你设计一个算法，计算男女运动员最佳配对法，使各组男女双方竞赛优势的总和达到最大。

输入格式

第一行有 1 个正整数 $n(1 \le n \le 20)$。接下来的 $2n$ 行，每行 n 个数。前 n 行是 P，后 n 行是 Q

输出格式

将计算出的男女双方竞赛优势的总和的最大值输出。

```
#include<bits/stdc++.h>
#define endl '\n'
#define int long long
using namespace std;
const int M=25;
int p[M][M];
int q[M][M];
int dp[(1<<20)+5];
```

```
void solve(){
    int n;
    cin>>n;
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            cin>>p[i][j];
        }
    }
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            cin>>q[i][j];
        }
    }
    for(int i=1;i<(1<<n);i++){
        int cnt=0;
        for(int j=1;j<=n;j++){
            if(i & (1<<j-1)) cnt++;
        }
        for(int j=1;j<=n;j++){
            if(i & (1<<j-1))
                dp[i]=max(dp[i],dp[i-(1<<j-1)]+q[cnt][j]*p[j][cnt]);
        }
    }
    cout<<dp[(1<<n)-1]<<endl;
    return ;
}
signed main()...
```

P2036 [COCI 2008/2009 #2] PERKET

题目描述

Perket 是一种流行的美食。为了做好 Perket，厨师必须谨慎选择食材，以在保持传统风味的同时尽可能获得最全面的味道。你有 n 种可支配的配料。对于每一种配料，我们知道它们各自的酸度 s 和苦度 b。当我们添加配料时，总的酸度为每一种配料的酸度总乘积；总的苦度为每一种配料的苦度的总和。

众所周知，美食应该做到口感适中，所以我们希望选取配料，以使得酸度和苦度的绝对差最小。

另外，我们必须添加至少一种配料，因为没有任何食物是只以水为配料的。

输入格式

第一行一个整数 n，表示可供选用的食材种类数。

接下来 n 行，每行 2 个整数 s_i 和 b_i，表示第 i 种食材的酸度和苦度。

输出格式

一行一个整数，表示可能的总酸度和总苦度的最小绝对差。

状压写法

```
#include<bits/stdc++.h>
#define endl '\n'
using namespace std;
int dp[(1<<15)+5];
struct food{
        int suan;
        int ku;
}foods[15];
int minn=0x3f3f3f3f;
void solve(){
        int n;
        cin>>n;
        for(int i=0;i<n;i++){
                cin>>foods[i].suan>>foods[i].ku;
        }
        for(int i=1;i<(1<<n);i++){
                int s=1,b=0;
                for(int j=0;j<n;j++){
                        if(1 &(i>>j)){
                                s*=foods[j].suan;
                                b+=foods[j].ku;
                        }
                }
                minn=min(minn,abs(s-b));
        }
        cout<<minn<<endl;
}
int main(){
        ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
        int _=1;
        //cin>>_;
        while(_--){
                solve();
        }
        return 0;
}
```

dfs写法

```
#include<bits/stdc++.h>
using namespace std;
int n,ming=1<<30,pl[15];
struct a{
    int suan;
    int ku;
}as[15];
void dfs(int k,int x,int y){
    if(k>n){
```

```
        if(y==0)
            return ;
        ming=min(abs(x-y),ming);
        return;
    }
    dfs(k+1,x,y);
    dfs(k+1,x*as[k].suan,y+as[k].ku);
    return ;
}
int main(){
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>as[i].suan>>as[i].ku;
    }
    dfs(1,1,0);
    cout<<ming;
    return 0;
}
```

数学算法（含高精度）

最大公约数 & 最小公倍数

输入两个正整数 _x_0,_y_0，求出满足下列条件的 P,Q 的个数：P,Q 是正整数 要求 P,Q 以 _x_0 为最大公约数，以 _y_0 为最小公倍数。 试求：满足条件的所有可能的 P,Q 的个数。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    int a,b,cnt=0;
    cin>>a>>b;
    for(int i=a;i<=b;i++){
        int j=a*b/i;
        if(gcd(i,j)==a&&(i*j)/gcd(i,j)==b) cnt++;
    }
    cout<<cnt<<endl;
    return 0;
}
```

线性筛素数

P3383 【模板】线性筛素数

题目描述

如题，给定一个范围 n，有 q 个询问，每次输出第 k 小的素数。

输入格式

第一行包含两个正整数 n, q，分别表示查询的范围和查询的个数。

接下来 q 行每行一个正整数 k，表示查询第 k 小的素数。

输出格式

输出 q 行，每行一个正整数表示答案。

```
#include<bits/stdc++.h>
using namespace std;
const long int M=1e8+10,N=6e6+10;
bool isP[M];
int P[N],cnt,n,q;
void check(int n){
    memset(isP,1,sizeof(isP));
    isP[1]=0;
    for(int i=2;i<=n;i++){
        if(isP[i])
            P[++cnt]=i;
        for(int j=1;j<=cnt&&i*P[j]<=n;j++){
            isP[i*P[j]]=0;
            if(i%P[j]==0) break;
        }
    }
}
signed main(){
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    cin>>n>>q;
    check(n);
    while(q--){
        int x;
        cin>>x;
        cout<<P[x]<<endl;
    }
    return 0;
}
```

高精度+

```
#include<iostream>
#include<vector>

using namespace std;
//C=A+B
vector<int> add(vector<int> &A,vector<int> &B)
{
    vector<int> C;
    int t=0; //t是进位
    for(int i=0;i<A.size() || i<B.size();i++)
    {
        if(i<A.size()) t+=A[i];
        if(i<B.size()) t+=B[i];
        C.push_back(t%10);
        t/=10;   //t若>=10，t/=10，t变为1，表明要进位，t若<10，表明不需进位，t变为0
    }
    if(t) C.push_back(1);  //如果最高位还有进位，在数组最后补1即可
    return C;
}

int main()
{
    string a,b;
    vector<int> A,B;

    cin>>a>>b;_//a="123456"
    for(int i=a.size()-1;i>=0;i--) A.push_back(a[i]-'0');//A=[6,5,4,3,2,1]
    for(int i=b.size()-1;i>=0;i--) B.push_back(b[i]-'0');

    vector<int> C=add(A,B);

    for(int i=C.size()-1;i>=0;i--) cout<<C[i];
    return 0;
}
```

高精度-

```
#include<iostream>
#include<vector>

using namespace std;
//判断是否有A>=B
bool cmp(vector<int> &A,vector<int> &B)
{
    if(A.size()!=B.size()) return A.size()>B.size();
    for(int i=A.size()-1;i>=0;i--)
        if(A[i]!=B[i])
            return A[i]>B[i];
    return true;
}
```

```
//C=A-B
vector<int> sub(vector<int> &A,vector<int> &B) //这里的A一定是需要大于等于B的才可以
{
    vector<int> C;
    for(int i=0,t=0;i<A.size();i++)
    {
        t=A[i]-t;
        if(i<B.size()) t-=B[i];
        C.push_back((t+10)%10);//若t<0，借位加10，若>=0，加10再余10还是t
        if(t<0) t=1; //借位
        else t=0;
    }
    while(C.size()>1 && C.back()==0) C.pop_back();
    //去多余的0，例如125-123=2，防止输出002
    return C;
}

int main()
{
   string a,b;
    vector<int> A,B;

    cin>>a>>b;_//a="123456"
    for(int i=a.size()-1;i>=0;i--) A.push_back(a[i]-'0');//A=[6,5,4,3,2,1]
    for(int i=b.size()-1;i>=0;i--) B.push_back(b[i]-'0');

    if(cmp(A,B))
    {
        vector<int> C=sub(A,B);
        for(int i=C.size()-1;i>=0;i--) cout<<C[i];
    }
    else       //若A<B时，用B-A再在前面加个负号_
   {
        vector<int> C=sub(B,A);
        cout<<"-";
        for(int i=C.size()-1;i>=0;i--) cout<<C[i];
    }
    return 0;
}
`
```

高精度X

```
#include<iostream>
#include<vector>
using namespace std;

//C=A*b
vector<int> mul(vector<int> &A,int b)
{
    vector<int> C;
    int t=0;_//进位
```

```
    for(int i=0;i<A.size() || t;i++)
    {
        if(i<A.size()) t+=A[i]*b;_//将b与A[i]相乘，而不是b的其中一位与A[i]相乘
        C.push_back(t%10);
        //相乘后只取最后一位的数_
        t/=10;
    }
    return C;
}

int main()
{
    string a;
    int b;
    cin>>a>>b;_//a="123456"
    vector<int> A;

    for(int i=a.size()-1;i>=0;i--) A.push_back(a[i]-'0');//A=[6,5,4,3,2,1]

    vector<int> C=mul(A,b);

    for(int i=C.size()-1;i>=0;i--) cout<<C[i];

    return 0;
}
```

高精度/

```
#include<iostream>
#include<vector>
#include<algorithm>

using namespace std;
//A/b,商是C，余数是r
 vector<int> div(vector<int> &A,int b,int &r)
{
    vector<int> C;
    r=0;
    for(int i=A.size()-1;i>=0;i--)
    {
        r=r*10+A[i];
        C.push_back(r/b);
        r%=b;
    }
    reverse(C.begin(),C.end());//反转，需头文件#include<algorithm>，例如12345变成
54321
    while(C.size()>1 && C.back()==0) C.pop_back();
    //去掉多余的0，比如89/9得到的是9余8，而不是09余8_
    return C;
}

int main()
```

```
{
    string a;
    int b;
    cin>>a>>b;_//a="123456"

    vector<int> A;
    for(int i=a.size()-1;i>=0;i--) A.push_back(a[i]-'0');//A=[6,5,4,3,2,1]

    int r;//余数
    vector<int> C=div(A,b,r);

    for(int i=C.size()-1;i>=0;i--) cout<<C[i];
    cout<<endl<<r<<endl;

    return 0;  }
```

异或

给定N个整数$A_1, A_2, A_3....A_N$中选出两个进行异或计算，得到的结果最大是多少

```
#include <bits/stdc++.h>
constexpr int N = 1e7 + 10;
int n;
int a[N];
int cnt;
struct EDGE
{
    int son[2];
} node[N];
void insert(int num)
{
    int now = 0;
    for (int i = 31; i >= 0; i--)
    {
        bool t = (num >> i) & 1;
        if (!node[now].son[t])
            node[now].son[t] = ++cnt;
        now = node[now].son[t];
    }
}
int find(int num)
{
    int ans = 0, now = 0;
    for (int i = 31; i >= 0; i--)
    {
        bool t = (num >> i) & 1;
        if (!node[now].son[t ^ 1])
            now = node[now].son[t];
        else
        {
            now = node[now].son[t ^ 1];
```

```
            ans = ans ^ (1 << i);
        }
    }
    return ans;
}
void solve()
{
    std::cin >> n;
    for (int i = 1; i <= n; i++)
    {
        std::cin >> a[i];
        insert(a[i]);
    }
    int sumxor = 0;
    for (int i = 1; i <= n; i++)
    {
        sumxor = std::max(sumxor, find(a[i]));
    }
    std::cout << sumxor << '\n';
}
signed main()
{
    std::ios::sync_with_stdio(0);
    std::cin.tie(0);
    int _ = 1;
//_ _std::cin_ _>>_ __;
    while (_--)
        solve();
    return 0;
}
```

求逆元(1~N)

```
#include <bits/stdc++.h>
using i64 = long long;
constexpr int N = 3e6 + 10;
i64 inv[N];
void solve()
{
    i64 n, p;
    std::cin >> n >> p;
    inv[1] = 1;
    for (int i = 2; i <= n; i++)
    {
        inv[i] = (p - p / i) * inv[p % i] % p;
    }
    for (int i = 1; i <= n; i++)
    {
        std::cout << inv[i] << '\n';
    }
}
signed main()
```

```
{
    std::ios::sync_with_stdio(0);
    std::cin.tie(0);
    int _ = 1;
//_ _std::cin_ _>>_ __;
    while (_--)
        solve();
    return 0;
}
```

差分数组

当你将原始数组中元素同时加上或者减掉某个数，那么他们的差分数组其实是不会变化的。

图论

最小生成树模板

P3366 【模板】最小生成树

题目描述

如题，给出一个无向图，求出最小生成树，如果该图不连通，则输出 `orz`。

输入格式

第一行包含两个整数 N, M，表示该图共有 N 个结点和 M 条无向边。

接下来 M 行每行包含三个整数 X_i, Y_i, Z_i，表示有一条长度为 Z_i 的无向边连接结点 X_i, Y_i。

输出格式

如果该图连通，则输出一个整数表示最小生成树的各边的长度之和。如果该图不连通则输出 `orz`。

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
int n,m,x,y,z,cnt;
```

```
ll ans;
const int M=2e5+10;
struct edge{
    int start,to;
    ll val;
}edges[M];
int f[M];
int find(int x){
    if(f[x]==x) return x;
    else{
        f[x]=find(f[x]);
        return f[x];
    }
}
bool cmp(edge a,edge b){return a.val<b.val;}
void kruskal(){
    for(int i=1;i<=m;i++){
        int u=find(edges[i].start);
        int v=find(edges[i].to);
        if(u==v) continue;
        ans+=edges[i].val;
        f[u]=v;
        cnt++;
        if(cnt==n-1) break;
    }
}

int main(){
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    cin>>n>>m;
    for(int i=1;i<=n;i++) f[i]=i;
    for(int i=1;i<=m;i++){
        cin>>edges[i].start>>edges[i].to>>edges[i].val;
    }
    sort(edges+1,edges+1+m,cmp);
    kruskal();
    if(find(1)!=find(n)){
        cout<<"orz";
        return 0;
    }
    cout<<ans;
    return 0;
}
```

最短路径算法（迪杰斯特拉）

```
struct Edge
{
    ll v,w,next;

//v:目的地,w:距离,next:下一个节点
}G[N];
```

```
ll head[N],cnt,n,m,s;
ll dis[N];//存距离
inline void addedge(ll u,ll v,ll w)

//链式前向星存图
{
    cnt++;
    G[cnt].w=w;
    G[cnt].v=v;
    G[cnt].next=head[u];
    head[u]=cnt;
}
struct node
{
    ll d,u;_//d是距离u是起点
    bool operator<(const node& t)const

//重载运算符
    {
        return d>t.d;
    }
};
inline void Dijkstra()
{
    for(register int i=1;i<=n;++i)dis[i]=mod;//初始化
    dis[s]=0;
    priority_queue<node>q;//堆优化
    q.push((node){0,s});//起点push进去
    while(!q.empty())
    {
        node tmp=q.top();q.pop();
        ll u=tmp.u,d=tmp.d;
        if(d!=dis[u])continue;

//松弛操作剪枝
        for(register int i=head[u];i;i=G[i].next)//链式前向星
        {
            ll v=G[i].v,w=G[i].w;
            if(dis[u]+w<dis[v])//符合条件就更新
            {
                dis[v]=dis[u]+w;
                q.push((node){dis[v],v});//沿着边往下走
            }
        }
    }
}
int main()
{
    scanf("%lld %lld %lld",&n,&m,&s);
    for(register int i=1;i<=m;++i)
    {
        ll x,y,z;
        scanf("%lld %lld %lld",&x,&y,&z);
        addedge(x,y,z);//建图
    }
    Dijkstra();
    for(register int i=1;i<=n;++i)
```

```
        printf("%lld ",dis[i]);
    printf("\n");
    return 0;
}
```

图的遍历

P3916 图的遍历 题目描述 给出 N 个点，M 条边的有向图，对于每个点 v，令 $A(v)$ 表示从点 v 出发，能到达的编号最大的点。现在请求出 $A(1), A(2), \ldots, A(N)$ 的值。 **输入格式** 第 1 行 2 个整数 N, M，表示点数和边数。 接下来 M 行，每行 2 个整数 U_i, V_i，表示边 (U_i, V_i)。点用 $1, 2, \ldots, N$ 编号。 **输出格式** 一行 N 个整数 $A(1), A(2), \ldots, A(N)$。

```
#include <iostream>
#include <cstdio>
#include <vector>
using namespace std;

#define MAXL 100010

int N, M, A[MAXL];
vector<int> G[MAXL]; //vector存图

void dfs(int x, int d) {
        if(A[x]) return; //访问过
        A[x] = d;
        for(int i=0; i<G[x].size(); i++)
                dfs(G[x][i], d);
}

int main() {
        int u, v;
        scanf("%d%d", &N, &M);
        for(int i=1; i<=M; i++) {
                scanf("%d%d", &u, &v);
                G[v].push_back(u); //反向建边
        }
        for(int i=N; i; i--) dfs(i, i);
        for(int i=1; i<=N; i++) printf("%d ", A[i]);
        printf("\n");
        return 0;
}
```

树的直径

```
const int N = 10000 + 10;
int n, c, d[N];
vector<int> E[N];
void dfs(int u, int fa)
{
    for (int v : E[u])
    {
        if (v == fa)
            continue;
        d[v] = d[u] + 1;
        if (d[v] > d[c])
            c = v;
        dfs(v, u);
    }
}
int main()
{
    scanf("%d", &n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        scanf("%d %d", &u, &v);
        E[u].push_back(v), E[v].push_back(u);
    }
    dfs(1, 0);
    d[c] = 0, dfs(c, 0);
    printf("%d\n", d[c]);
    return 0;
}
```

树形DP

```
const int N = 10000 + 10;
int n, d = 0;
int d1[N], d2[N];
vector<int> E[N];
void dfs(int u, int fa)
{
    d1[u] = d2[u] = 0;
    for (int v : E[u])
    {
        if (v == fa)
            continue;
        dfs(v, u);
        int t = d1[v] + 1;
        if (t > d1[u])
            d2[u] = d1[u], d1[u] = t;
        else if (t > d2[u])
            d2[u] = t;
    }
```

```
    d = max(d, d1[u] + d2[u]);
}
int main()
{
    scanf("%d", &n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        scanf("%d %d", &u, &v);
        E[u].push_back(v), E[v].push_back(u);
    }
    dfs(1, 0);
    printf("%d\n", d);
    return 0;
}
```

LCA

(最近公共祖先)

```
#include <bits/stdc++.h>
#define int long long
constexpr int N = 5e5 + 10;
std::vector<int> e[N];
int fa[N], dep[N], son[N], sz[N], top[N]; _//_ _分别存储节点i的父节点，深度，重儿子，
以i为根的子树的节点数，i所在重链的顶点
int n, m, s;
void dfs1(int u, int father)
{ //_ _找出每个点的父节点，深度，和重儿子，以及以每个节点为根的子树的节点数
    fa[u] = father;
    dep[u] = dep[father] + 1;
    sz[u] = 1;
    for (auto to : e[u])
    {
        if (to == father)
            continue;
        dfs1(to, u);
        sz[u] += sz[to];
        if (sz[son[u]] < sz[to])
            son[u] = to;
    }
}
void dfs2(int u, int t)
{
    top[u] = t;
    if (son[u] == 0)
        return;
    dfs2(son[u], t);
    for (auto to : e[u])
```

```
    {
        if (to == fa[u] || to == son[u])
            continue;
        dfs2(to, to);
    }
}
int LCA(int u, int v)
{
//_ _当u,v在同一个重链上面的时候，较小的那个就是最近公共祖先
    while (top[u] != top[v])
   {
        if (dep[top[u]] < dep[top[v]])
            std::swap(u, v); _//_ _保证top[u]比top[v]深
        u = fa[top[u]];
    }
    return dep[u] < dep[v] ? u : v;
}
void input()
{
    std::cin >> n >> m >> s;
    for (int i = 1; i < n; i++)
    {
        int x, y;
        std::cin >> x >> y;
        e[x].push_back(y), e[y].push_back(x);
    }
}
void ask()
{
    for (int i = 1; i <= m;i++)
    {
        int a, b;
        std::cin >> a >> b;
        std::cout << LCA(a, b) << std::endl;
    }
}
void solve()
{
    input();
    dfs1(s, 0);
    dfs2(s, s);
    ask();
}
int main.......
```

拓扑排序

```
#include <bits/stdc++.h>
constexpr int MAXN = 1e3 + 10;
int n, m;
std::vector<int> edge[MAXN];
int inna[MAXN];
```

```
bool inque[MAXN];
void input()
{
    std::cin >> n >> m;
    for (int i = 1; i <= m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        edge[u].push_back(v);
        inna[v]++;
    }
}
void toposort()
{
    std::queue<int> q;
    for (int i = 1; i <= n; i++)
    {
        if (inna[i] == 0)
            q.push(i), inque[i] = 1;
    }
    while (!q.empty())
    {
        int now = q.front();
        q.pop();
        std::cout << now << " ";
        for (const auto &to : edge[now])
        {
            inna[to]--;
            if (!inna[to] && !inque[to])
            {
                q.push(to);
            }
        }
    }
}
void solve()
{
    std::cin >> n;
    for (int i = 1; i <= n; i++)
    {
        while (1)
        {
            int x;
            std::cin >> x;
            if (x == 0)
                break;
            else
            {
                edge[i].push_back(x);
                inna[x]++;
            }
        }
    }
    toposort();
}
signed main()
{
```

```
    std::ios::sync_with_stdio(0);
    std::cin.tie(0);
    solve();
    return 0;
}
```

强连通分量

```
#include <iostream>
#include <vector>
using namespace std;
const int N = 10005;
vector<int> e[N]; //_ _存边
int dfn[N], low[N], tot;//_ _时间戳，追溯值，时间戳的编号
int stk[N], instk[N], top; _//_ _栈，是否在栈中，栈指针
int scc[N], siz[N], cnt; //_ _记录某个点在哪个强连通分量中，某个强连通分量的点数，强连通
分量的编号
int n, m; //_ _n__个点，m条边
int ans = 0;
void tarjan(int x)
{
//_ _开始搜索x
    dfn[x] = low[x] = ++tot;
    stk[++top] = x, instk[x] = 1;
    for (int y : e[x])
    {
        if (!dfn[y])
        { //_ _如果y尚未被访问
            tarjan(y);
            low[x] = min(low[x], low[y]); //_ _回x的时候更新low
        }
        else if (instk[y])
        { //_ _若y已经被访问且在栈中
            low[x] = min(low[x], dfn[y]); //_ _更新low
        }
    }
    if (dfn[x] == low[x])
    {
        int y;
        cnt++;
        while (1)
        {
            y = stk[top--];
            instk[y] = 0;
            scc[y] = cnt; //_ _记录y隶属于哪一个强连通分量
            ++siz[cnt]; //_ _记录这个强连通分量的大小
            if (y == x)
            break;
        }
    }
}
void init()
```

```
{
    cin >> n >> m;
    for (int i = 1; i <= m; i++)
    {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v);
    }
}
int main()
{
    init();
    for (int i = 1; i <= n; i++)
        if (!dfn[i])
            tarjan(i);
    for (int i = 1; i <= cnt; i++)
        if (siz[i] > 1)
            ans++;
    cout << ans << endl;
    return 0;
}
```

Floyd（全源最短路）

```
//_ _Floyd
#include <bits/stdc++.h>
constexpr int MAXN = 1e2 + 10;
int n, m;
int adj[MAXN][MAXN], dis[MAXN][MAXN];
bool vis[MAXN];
void input()
{
    memset(adj, 0x3f, sizeof(adj));
    memset(dis, 0x3f, sizeof(dis));
    std::cin >> n >> m;
    for (int i = 1; i <= n; i++)
        adj[i][i] = 0;
    for (int i = 1; i <= m; i++)
    {
        int u, v, val;
        std::cin >> u >> v >> val;
        adj[u][v] = std::min(adj[u][v], val), adj[v][u] = std::min(adj[v][u], val);
    }
}
void Floyd()
{
    for (int u = 1; u <= n; u++)
    {
        for (int v = 1; v <= n; v++)
        {
            dis[u][v] = adj[u][v];
```

```
        }
    }
    for (int k = 1; k <= n; k++)
    {
        for (int u = 1; u <= n; u++)
        {
            for (int v = 1; v <= n; v++)
            {
                dis[u][v] = std::min(dis[u][v], dis[u][k] + dis[k][v]);
            }
        }
    }
}
void output()
{
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
            std::cout << dis[i][j] << " ";
        std::cout << std::endl;
    }
}
void solve()
{
    input();
    Floyd();
    output();
}
signed main()
{
    std::ios::sync_with_stdio(0);
    std::cin.tie(0);
    solve();
    return 0;
}
```

数据结构 STL 特殊库函数

queue

1.push() 在队尾插入一个元素

2.pop() 删除队列第一个元素

3.size() 返回队列中元素个数

4.empty() 如果队列空则返回true

5.front() 返回队列中的第一个元素

6.back() 返回队列中最后一个元素

priority_queue

(默认大到小)

```
priority_queue <node> q;//结构体里重载了‘<’小于符号`
priority_queue <int,vector<int>,greater<int> > q;//注意后面两个“>”不要写在一起，“>>”是右移运算符,小到大
priority_queue <int,vector<int>,less<int> >q;//大到小
q.size();//返回q里元素个数
q.empty();//返回q是否为空，空则返回1，否则返回0
q.push(k);//在q的末尾插入k
q.pop();//删掉q的第一个元素
q.top();//返回q的第一个元素
```

set

insert(元素): 插入一个元素。 erase(元素): 删除一个元素。 find(元素): 查找一个元素。 begin(),返回set容器的第一个元素 end(),返回set容器的最后一个元素 clear(),删除set容器中的所有的元素 empty(),判断set容器是否为空 max_size(),返回set容器可能包含的元素最大个数 size(),返回当前set容器中的元素个数 rbegin,返回的值和end()相同 rend(),返回的值和rbegin()相同 **set的输出不支持访问下标**

```
for(const auto& person :q){ cout<<person.name<<endl; }
```

vector

(记得开空间)

```
a.assign(b.begin(), b.begin()+3); //b为向量，将b的0~2个元素构成的向量赋给a
a.assign(4,2); //是a只含4个元素，且每个元素为2
a.back(); //最后一个元素
a.front(); //第一个元素
a[i]; //返回a的第i个元素，当且仅当a[i]存在
a.clear(); //清空
a.empty(); //判断a是否为空，空则返回ture,不空则返回false
```

```
a.pop_back(); //删除a向量的最后一个元素
a.erase(a.begin()+1,a.begin()+3); //删除a中第1个（从第0个算起）到第2个元素，也就是说删除的元素从a.begin()+1算起（包括它）一直到a.begin()+3（不包括它）
a.push_back(5);
a.insert(a.begin()+1,5); //在a的第1个元素（从第0个算起）的位置插入数值5，如a为1,2,3,4，插入元素后为1,5,2,3,4
a.insert(a.begin()+1,3,5); //在a的第1个元素（从第0个算起）的位置插入3个数，其值都为5
a.insert(a.begin()+1,b+3,b+6); //b为数组，在a的第1个元素（从第0个算起）的位置插入b的第3个元素到第5个元素（不包括b+6），如b为1,2,3,4,5,9,8          ，插入元素后为1,4,5,9,2,3,4,5,9,8
a.size();
a.capacity(); //返回a在内存中总共可以容纳的元素个数
a.resize(10); //将a的现有元素个数调至10个，多则删，少则补，其值随机
a.resize(10,2); //将a的现有元素个数调至10个，多则删，少则补，其值为2
a.reserve(100); //将a的容量（capacity）扩充至100，也就是说现在测试a.capacity();的时候返回值是100.这种操作只有在需要给a添加大量数据的时候才显得有意义，因为这将避免内存多次容量扩充操作（当a的容量不足时电脑会自动扩容，当然这必然降低性能）
a.swap(b);
a==b; //b为向量，向量的比较操作还有!=,>=,<=,>,<
```

map

```
map<node,node> m;
begin()    返回指向map头部的迭代器`
clear() 删除所有元素`
count() 返回指定元素出现的次数**
empty() 如果map为空则返回true**
end() 返回指向map末尾的迭代器**
equal_range() 返回特殊条目的迭代器对**
erase() 删除一个元素
find() 查找一个元素
get_allocator() 返回map的配置器
insert() 插入元素
key_comp() 返回比较元素key的函数
lower_bound() 返回键值>=给定元素的第一个位置
max_size() 返回可以容纳的最大元素个数
rbegin() 返回一个指向map尾部的逆向迭代器
rend() 返回一个指向map头部的逆向迭代器
size() 返回map中元素的个数
swap() 交换两个map
upper_bound() 返回键值>给定元素的第一个位置
value_comp() 返回比较元素value的函数
```

插入数据:

```
        cout << "第一种方法插入 ! " << endl;
        m.insert(pair<int ,string>(1,"stu1"));
        m.insert(pair<int ,string>(2,"stu2"));
```

```
        m.insert(pair<int ,string>(3,"stu3"));
        map<int,string>::iterator t;
        for( t = m.begin(); t != m.end(); t++ )   cout << t->first << ' ' << t-
>second << endl;//t->first表示关键字，t->second 表示键值
        cout << "****************************" << endl;
        cout << "第二种方法插入 ! " << endl;
        m[1] = "stu1";
        m[2] = "stu2";
        m[3] = "stu3";
        for( t = m.begin(); t != m.end(); t++ )   cout << t->first << ' ' << t-
>second << endl;
```

检查键是否存在:

```
if (myMap.find(key) != myMap.end()) {// 键存在}
```

删除元素:

```
myMap.erase(key);
```

清空 map:

```
myMap.clear();
```

获取 map 的大小:

```
size_t size = myMap.size();
std::map<int, int, std::greater<int> > m;
//map默认的递增，这样改成递减的
```

要按照键值排序则: 假设已有一组map<string,int>类型的数mp,则具体实现过程为：

```
vector< pair<string,int> > vec;
for(map<string,int>::iterator it = mp.begin(); it != mp.end(); it++)
        vec.push_back( pair<string,int>(it->first,it->second) );
sort(vec.begin(),vec.end(),cmp);
bool cmp(pair<string,int> a, pair<string,int> b)
        return a.second < b.second;
```

stack

```
//声明
stack<int> s;
stack<string> s;
stack<***> s;
```

```
s.push(e)         //元素e入栈
s.pop()       //移除栈顶元素
s.top()       //取得栈顶元素（但不删除）
s.empty()         //检测栈内是否为空，空为真
s.size()          //返回栈内元素的个数
```

pair

```
//1.初始化定义
pair<string, int> p("wangyaqi",1);//带初始值的
pair<string, int> p;//不带初始值的
//2.赋值
p = {"wang", 18};
p = make_pair("wang", 18);
p = pair<string, int>("wang", 18);
//3.定义结构体数组
pair<int,int> p[20];
```

string

at() 访问字符串中指定位置的字符（带边界检查）。

```
std::cout << str.at(0);

substr()        返回从指定位置开始的子字符串。
std::string sub = str.substr(0, 5);

find()  查找子字符串在字符串中的位置。
std::cout << str.find("sub") << std::endl;

rfind() 从字符串末尾开始查找子字符串的位置。
std::cout << str.rfind("sub") << std::endl;
```

```
replace()        替换字符串中的部分内容。
str.replace(pos, length, "new_substring");

append()         在字符串末尾添加内容。
str.append(" more");

insert()         在指定位置插入内容。
str.insert(pos, "inserted");

erase() 删除指定位置的字符或子字符串。
str.erase(pos, length);

clear() 清空字符串。
str.clear();

data()  返回指向字符数据的指针（C++11 及之后的版本）。
const char* data = str.data();

compare()        比较两个字符串。
int result = str.compare("other");

find_first_of() 查找第一个匹配任意字符的位置。
size_t pos = str.find_first_of("aeiou");

find_last_of()  查找最后一个匹配任意字符的位置。
size_t pos = str.find_last_of("aeiou");

find_first_not_of()      查找第一个不匹配任意字符的位置。
size_t pos = str.find_first_not_of("aeiou");

find_last_not_of()       查找最后一个不匹配任意字符的位置。
size_t pos = str.find_last_not_of("aeiou");
```

线段树

基础线段树

题目描述

如题，已知一个数列 $\{a_i\}$，你需要进行下面两种操作：

1. 将某区间每一个数加上 k。
2. 求出某区间每一个数的和。

输入格式

第一行包含两个整数 n, m，分别表示该数列数字的个数和操作的总个数。

第二行包含 n 个用空格分隔的整数 a_i，其中第 i 个数字表示数列第 i 项的初始值。

接下来 m 行每行包含 3 或 4 个整数，表示一个操作，具体如下：

1. `1 x y k`：将区间 $[x, y]$ 内每个数加上 k。
2. `2 x y`：输出区间 $[x, y]$ 内每个数的和。

```
#include<bits/stdc++.h>
#define endl '\n'
#define int long long
using namespace std;
const int M=1e5+10;
int num[M];
int sum[4*M];//线段树：和
int ad[4*M];//懒数组
int n,m;
void up(int i);
void down(int i,int ln,int rn);

//懒数列改动
void lazy(int i,int v,int n){
        sum[i]+=v*n;
        ad[i] +=v;
}

//递归建树
void  creat(int l,int r,int i){
        if(l==r){
                sum[i]=num[l];
        }
        else{//左右中，后序遍历
                int mid=l+(r-l)/2;//严格从1~n，每次从中点mid划分
                creat(l,mid,i << 1);//i*2
                creat(mid+1,r,i<<1 | 1);//i*2+1
                up(i);
        }
        ad[i]=0;
}

//查询函数
int  check(int nowl,int nowr,int l,int r,int i){//每次任务中nowl，nowr固定不变，在变化的l和r区间中寻找
        if(nowl <= l && r<=nowr){
                return sum[i];//如果目标区间小于任务区间，直接选择
        }
        int mid=l+(r-l)/2;
        down(i,mid-l+1,r-mid);
        int ans=0;
        if(nowl <= mid ){
                ans+=check(nowl,nowr,l,mid,i<<1);
        }
        if(nowr >mid){
                ans+=check(nowl,nowr,mid+1,r,i<<1 | 1);
        }
```

```
        return ans;
}

void add(int nowl,int nowr,int v,int l,int r,int i){
        if(nowl <= l && r<=nowr){//全包则不下发，直接修改大的，其余进入懒数列待处理
                lazy(i,v,r-l+1);
        }
        else{
                int mid=l+(r-l)/2;
                down(i,mid-l+1,r-mid);//优先处理待处理的懒数列
                if(nowl <=mid){//不全包则下发
                        add(nowl,nowr,v,l,mid,i<<1);
                }
                if(nowr >mid){
                        add(nowl,nowr,v,mid+1,r,i<<1 | 1);
                }
                up(i);
        }
}

void solve(){
        int _;
        cin>>_;
        if(_==1){
                int x,y,k;
                cin>>x>>y>>k;
                add(x,y,k,1,n,1);
        }
        else{
                int x,y;
                cin>>x>>y;
                cout<<check(x,y,1,n,1)<<endl;
        }
}

//汇总重新处理
void up(int i){
        sum[i]=sum[i<<1]+sum[i<<1 | 1];
}

//下发懒数列
void down(int i,int ln,int rn){//ln,rn指l~mid的长度
        if(ad[i]!=0){
                lazy(i<<1 ,ad[i],ln);
                lazy(i<<1 |1 ,ad[i],rn);
                ad[i]=0;
        }
}

signed main(){
        ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
        cin>>n>>m;
        for(int i=1;i<=n;i++){
                cin>>num[i];
        }
        creat(1,n,1);
        while(m--){
```

```
                solve();
        }
        return 0;
}


/*
5 5
1 5 4 2 3
2 2 4
1 2 3 2
2 3 4
1 1 5 1
2 1 4

11
8
20
*/
```

memset函数

例如：

```
typedef A[2];
memset(A, value, sizeof A);
(1)若str指向char型地址，value可为任意字符值；
(2) 若str指向非char型，如int型地址，要想赋值正确，value的值只能是-1或0
(3)初始化为无穷大
memset(a , 0x3f , sizeof a);
```

数学小妙招

1. $$(a-b)\equiv 0 \pmod{c}$$

   $$a\equiv b \pmod{c}$$

2. $longlong$的最大范围在$5*10^{14}$
3. 2^{60}刚好大于10^{18}
4. **数论分块的具体过程:** 计算和式 $\sum_{i=1}^{n} f(i)g(\lfloor\frac{n}{i}\rfloor)$的取值， 可以依据$\lfloor\frac{n}{i}\rfloor$的取值将标号$i=1,2,3...n$分块，因为$\lfloor\frac{n}{i}\rfloor$取值相同的标号是一段连续整数$[l,r]$,所以该块中和式的取值为$(\sum_{i=1}^{n} f(i))g(\lfloor\frac{n}{i}\rfloor)$. 为了快速计算该和式，通常需要能够快速计算左侧关于f的和式． 有些时候，该和式的表达式是已知的，可以在$O(1)$时间内完

成单次计算；有些时候，可以预处理出它的前缀和，仍然可以在$O(1)$时间内完成单次查询.

顺次计算每块左右端点时，当前块的左端点l就等于前一块的右端点再加1,而当前块的右端点就等于$\lfloor \frac{n}{\lfloor \frac{n}{l} \rfloor} \rfloor$ 由此，可以得到如下伪代码：

Algorithm Sum(f, g, n) :

Input. $n,\ s(k) = \sum_{i=1}^{k} f(k),\ g(k)$.

Output. $S(n) = \sum_{i=1}^{n} f(i) g(\lfloor n/i \rfloor)$.

Method.

$l \leftarrow 1$

result $\leftarrow 0$

while $l \leq n$ do

$r \leftarrow \lfloor \frac{n}{\lfloor n/l \rfloor} \rfloor$

result $\leftarrow$ result $+ (s(r) - s(l-1)) \cdot g(\lfloor \frac{n}{l} \rfloor)$

$l \leftarrow r + 1$

end while

return result

假设单次计算$s(\cdot)$的时间复杂度为$O(1)$则整个过程的时间复杂度为$O(n)$